Assignment 5 - Strategy Pattern

Ramin Smagulov | SE-2529 | Software Design Patterns | 2026

1 Introduction

I chose Option B, a calculator with interchangeable arithmetic algorithms. The client selects addition, subtraction, or multiplication and passes that operation to the Calculator. The context delegates each calculation through the common CalculationStrategy interface, and can receive a different strategy at runtime.

Each concrete strategy contains one operation. The client selects strategies directly; Calculator does not inspect their types or contain an if/switch chain. The demo applies each strategy to 12 and 4.

2 UML Class Diagram

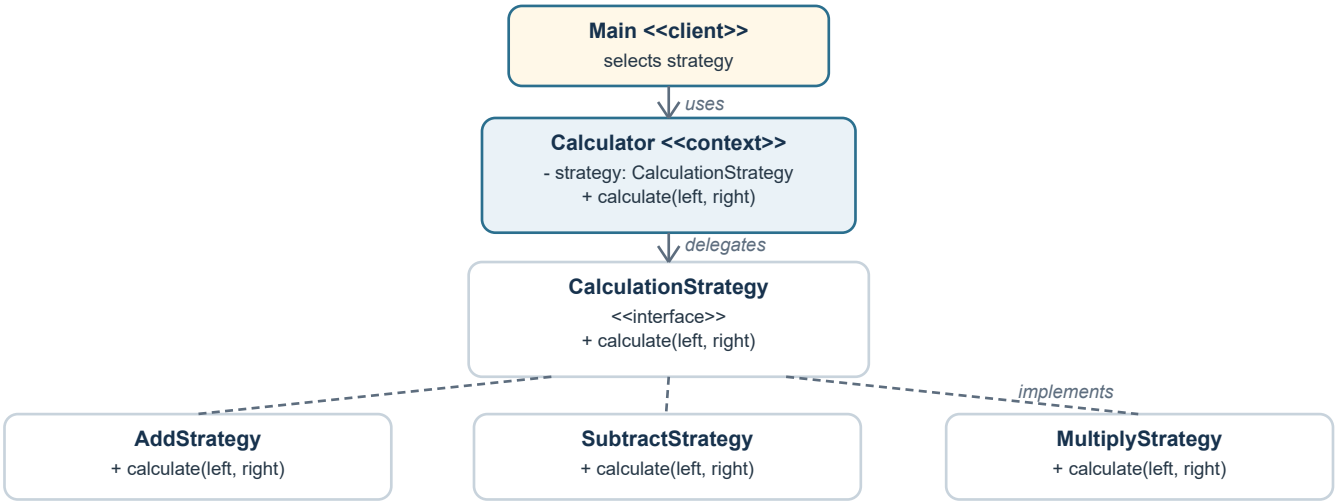


Figure 1. Main selects a strategy through Calculator. Calculator delegates to the interface; each concrete operation implements the same contract.

Pattern role	Class	Responsibility
Context	Calculator	Stores a strategy and delegates calculate().
Strategy	CalculationStrategy	Declares the shared calculation method.
Concrete strategies	Add / Subtract / MultiplyStrategy	Implements one arithmetic operation.
Client	Main	Selects strategies and shows runtime switching.

3 Clean Code

Replace branching with polymorphism

Calculator has no operation conditional. It sends the request to the selected implementation.

```
public double calculate(double left, double right) {
    return strategy.calculate(left, right);
}
```

Use intention-revealing names

Names communicate the algorithm and the operands, so callers do not pass unexplained values.

```
public final class SubtractStrategy
        implements CalculationStrategy {
    public double calculate(double left, double right) {
```

Give each strategy one responsibility

AddStrategy owns only addition. Subtraction and multiplication each live in their own class.

```
public double calculate(double left, double right) {
    return left + right;
}
```

Depend on an abstraction

Calculator stores the interface, so it does not need to know concrete strategy details.

```
private CalculationStrategy strategy;
```

Keep the context open to extension

Main can select another implementation at runtime without editing Calculator.

```
calculator.setStrategy(new SubtractStrategy());
printResult("Subtraction", left, right, calculator);
```

4 Strategy vs. State

This calculator uses Strategy because the client deliberately chooses an arithmetic algorithm for an operation and may replace it at any time. The strategies do not represent stages in the calculator's lifecycle and do not trigger transitions among themselves. A State design would fit an object whose behavior changes as its internal state advances; that is not the problem modeled here.

5 Verification

I compiled the Java sources with javac --release 17 and ran StrategyChecks. The checks compare the three arithmetic results, replace strategies on one Calculator instance, and verify that a null strategy is rejected without replacing the current one. All five checks passed.

Check	Expected behavior	Result
T1	12 + 4 = 16	PASS
T2	12 - 4 = 8	PASS
T3	12 × 4 = 48	PASS
T4	One Calculator switches among all strategies	PASS
T5	Null rejected; current strategy remains active	PASS

SUMMARY: 5/5 PASS

6 Conclusion

Strategy makes the calculator flexible: the client can change the arithmetic algorithm without changing the context. The trade-off is one class per operation plus a shared interface. For this small example that adds a few files, while keeping each algorithm isolated and making a new operation easy to add.

7 Repository

https://github.com/Ramin293/sdp_assignment5

8 References

[1] Astana IT University. Assignment #5 - Strategy Pattern. Course handout, 2026-2027, Sections 1-5 and 8.

[2] Freeman, E. and Robson, E. Head First Design Patterns, 2nd ed., Chapter 1: Strategy. O'Reilly, 2021.

[3] Martin, R. C. Clean Code, Chapter 10. Prentice Hall, 2008.

[4] Refactoring.Guru. Strategy design pattern. https://refactoring.guru/design-patterns/strategy